Estimating Dewpoint Temperature for Peru Potato Planting Locations Using an Anomaly Local Linear Regression Model

Piero Palacios Henry Juarez

What is this project?

'tdew_estimation' is a scientific software tool that estimates **daily dewpoint temperature** (often called "Td" or "td") in places or periods where that measurement is missing. Dewpoint temperature is a key climate variable describing the temperature at which air becomes saturated with moisture and dew can form. This tool was built to support **Simcast workflows** (for example, in agriculture or environmental science), where having dewpoint data is essential but direct measurements may be unavailable.

What is it for?

Many scientific applications—like forecasting plant disease risks, running environmental models, or filling out weather records—need complete, accurate records of daily dewpoint temperature for different locations (called "grid cells"). However, not all weather stations measure dewpoint every day; sometimes, important days or locations are missing this measurement.

This tool helps by **reconstructing or estimating** the missing values using a statistical model. In one sentence: it learns, separately for every location and every time of year, how the dewpoint *departs from its normal value* whenever the minimum temperature departs from *its* normal value—and then uses that learned relationship to fill in the missing dewpoint from the (usually available) minimum-temperature record.

The data come from **PISCOt**, a gridded daily climate product for Peru. The full national grid holds roughly **2 million cells (`IDs`)**, but this project does not need all of them: it restricts the work to the **Peru potato-planting zones, about 302,000 cells** (the default `--peru-potato` domain in the run scripts). Running the full ~2M grid is possible but is about **6.6× the work**. All of this is done at high spatial resolution and specifically for cases where dewpoint is missing or has gaps.

The statistical model: what we estimate and why

This section is written for readers comfortable with regression. The goal is to estimate **daily dewpoint temperature** TD at locations and on days where it was not measured, using minimum temperature $TMIN$ (which is far more widely recorded) plus the recent history of both variables.

Step 1 — Remove the seasonal cycle (climatology)

Dewpoint and temperature both have a strong, predictable annual cycle: a January day and a July day are not comparable. Regressing the raw values would mostly be re-discovering the calendar. So the first step is to estimate, for **each location** ID and **each day-of-year** $doy \in \{1, \ldots, 366\}$, the long-run mean over all available years:

$$\overline{TD}(ID, doy), \qquad \overline{TMIN}(ID, doy).$$

This per-location, per-day mean is the **climatology**—the "normal" value. Think of it as a location-specific seasonal baseline estimated from decades of data.

Step 2 — Model the anomalies, not the raw values

We then subtract the baseline and work with **anomalies** (departures from normal):

$$TD_{\text{anom}} = TD - \overline{TD}(ID, doy), \qquad TMIN_{\text{anom}} = TMIN - \overline{TMIN}(ID, doy).$$

After this de-seasonalizing step, the remaining signal is "warmer/wetter than usual" vs. "cooler/drier than usual"—exactly the part that minimum temperature can help explain. This is the standard *anomaly* framing used throughout climatology.

Step 3 — A local weighted linear regression per location and season

For each (ID, doy) pair we fit a small **weighted least-squares (WLS)** regression with five terms:

$$TD_{\text{anom}} \;\approx\; \beta_0 \;+\; \beta_1\, TMIN_{\text{anom}} \;+\; \beta_2\, TD^{(t-1)}_{\text{anom}} \;+\; \beta_3\, TD^{(t-2)}_{\text{anom}} \;+\; \beta_4\, TMIN^{(t-1)}_{\text{anom}},$$

where the superscripts $(t-1)$ and $(t-2)$ are **lags** (yesterday's and the day-before's anomalies). The lags capture short-term persistence: humid spells and cold snaps last several days, so recent conditions carry information about today.

Two design choices make this *local* rather than one global regression:

- **A window around the target day.** Only observations whose day-of-year falls within a half-width of $h = 11$ days of the target *doy* (wrapping around the calendar, so late-December borrows from early-January) enter the fit. A model for July 15th is therefore trained mostly on early-to-late-July data across all years—not on April or November.
- **Tricube weighting.** Within that window, observations are *weighted* by a **tricube kernel**: days right on the target get full weight, and weight falls smoothly to zero at the edge of the window. This is the same idea as LOESS smoothing—nearby points matter most. The analogy: rather than asking "what's the average relationship over the whole year," we ask "what's the relationship *right around this time of year*, leaning hardest on the closest days."

The result is **one set of five coefficients** $(\beta_0, \ldots, \beta_4)$ **for every** (ID, doy) **pair**, plus an R^2. To predict a missing dewpoint, the tool re-attaches the baseline: $\widehat{TD} = \overline{TD}(ID, doy) + \widehat{TD}_{\text{anom}}$.

The training period and how we check it out-of-sample

The model is **trained on the years 1981–2016** — that is **36 years** of daily PISCOt records (the `--train-start 1981 --train-end 2016` setting used throughout the run scripts). The production model uses the **whole** 1981–2016 record for every cell, because the goal is the most stable possible coefficients before applying them to fill gaps or to forecast later years (e.g. 2017 onward).

To judge whether the estimates are actually any good, we validate **out-of-sample in time**: fit on an early stretch of years and predict a *held-out* later stretch the model never saw. A clean way to report skill is a split such as **train 1981–2001, test 2002–2016**; in the repository's TMIN-version comparison the same idea is applied as train-through-2014 / forecast-2015. The principle is the standard one — never score a model on the same days it was fitted on.

Why this is a *big* estimation problem

This is where the statistics meets the computing. Two different "large numbers" are at play, and it is worth keeping them apart:

- **How many regressions?** The model is not one regression — it is an enormous *family of tiny, independent regressions*, **one per `(ID, doy)` pair**. The year axis does **not** multiply this count (the years are the *sample* inside each fit, not separate fits):

$$\underbrace{3.02 \times 10^5}_{\text{potato cells}} \times \underbrace{366}_{\text{days of year}} \approx 1.1 \times 10^8 \text{ separate } 5 \times 5 \text{ WLS fits.}$$

- **How much data feeds them?** Each fit pools 36 years of daily observations (within its $\pm h$ day window). Counting every daily record that must be read, de-seasonalized, lagged and weighted:

$$\underbrace{3.02 \times 10^5}_{\text{cells}} \times \underbrace{366}_{\text{days}} \times \underbrace{36}_{\text{years (1981–2016)}} \approx 4 \times 10^9 \text{ daily observations.}$$

So we solve ~**110 million** independent regressions, fed by ~**4 billion** daily data points. Each fit is trivial on its own (a 5-parameter regression solves in microseconds), but the sheer count — and the fact that every fit is fully independent of the others — is exactly what makes the next sections (Dask, HPC, CPU vs. GPU) necessary. (On the full ~2M-cell PISCOt grid these numbers grow by the ~6.6× factor mentioned earlier, to roughly 7×10^8 fits and 2.7×10^{10} observations.)

Why we parallelize: Dask and HPC

A useful way to see the problem: it is **"embarrassingly parallel."** Because the regression for, say, location A on July 15th does not depend in any way on the regression for location B on March 3rd, all ~110 million fits *could* in principle be computed at the same time. Nothing forces them into a sequence. The only reason a single laptop is slow is that it has to work through them more or less one at a time.

Running them one-by-one would take **weeks** on even a fast computer—and a laptop would likely run out of memory loading decades of nationwide data and crash. The work is not hard; there is just an enormous *amount* of it. This is precisely the situation where parallel computing pays off.

Dask: a manager that hands out the work

Dask is the Python library we use to split the job up and farm it out. The mental model is an office with one manager and many workers:

- We first cut the country's locations into **buckets**—contiguous groups of locations bundled together (so related data is read from disk once, not re-opened thousands of times).
- Each bucket becomes **one Dask task**: "fit every (ID, doy) regression for these locations."
- Dask hands tasks out to a pool of **workers** running in parallel and combines the results at the end.

The code (`run_bucketed_anomaly_training_dask`) is deliberately robust for long runs on shared machines:

- **A sliding window of tasks in flight.** Rather than dumping all buckets on the scheduler at once, it keeps a fixed number (a `batch_size`) in progress and feeds in new ones as others finish—"a slow bucket never blocks the rest of the fleet."

- **Failure isolation.** If one worker dies (a frequent reality on a busy cluster), that single bucket is logged and skipped while everyone else keeps going; a follow-up "patch" run cleans up the gaps.
- **No thread fighting.** Each worker is pinned to a single math (BLAS) thread, so workers don't trip over each other competing for the same CPU cores.

HPC: where Dask actually runs

A laptop has a handful of workers; we need *many*. So the tool is designed to run on **HPC (High-Performance Computing) clusters**—the kind of shared supercomputer a research institute operates, where a job is submitted to a queue (via a scheduler called **SLURM**) and granted dozens of CPU cores, or a slice of a high-end GPU, for a few hours. Our runs target the **KHIPU** cluster.

What this means in practice:

- The computation **does not run on a normal laptop**; it needs access to HPC resources, set up once by a data manager.
- With a cluster, a job that would take weeks finishes in **hours**.
- This is the same strategy national weather agencies use to run large climate models—divide the globe into pieces, compute the pieces simultaneously.

CPU vs. GPU: two ways to do the same math

A central engineering question for this project is *which kind of hardware should do the 110 million fits.* We built **two computational paths that produce numerically identical results** (both use 64-bit precision; outputs match to the byte). The shared statistical model is the same—only the engine differs.

The CPU path — many workers, each steady and sequential

On CPUs we use the standard, trusted route: a worker fits the regressions in its bucket **one at a time** using the well-known `statsmodels` weighted-least-squares routine. A CPU core is like a **highly skilled expert**: it does any single regression carefully and quickly, but it can only do one at a time. Speed comes from having **many experts (workers) in parallel**—on KHIPU, typically up to 32 CPU cores per job, each chewing through its own pile of buckets.

The GPU path — one device, thousands of fits at once

A GPU (we target one slice of an **NVIDIA A100**) works on a completely different principle. Instead of a few skilled experts, it has **thousands of simple workers** that are only useful when they all do the *same kind* of small task simultaneously. The analogy: a CPU is a handful of PhD statisticians; a GPU is a stadium full of students each handed one tiny identical worksheet.

To exploit that, the GPU path (`gpu_train.py`) does not loop over fits. It **batches** them: it assembles the ingredients for *all* ~100,000 fits in a bucket at once, then solves them together in a single fused GPU routine (a custom CUDA kernel that runs a tiny 5×5 Cholesky solve, one fit per GPU thread). Tens of thousands of regressions are cleared in **milliseconds** per wave.

The trade-off, in plain terms:

	CPU	**GPU**
Style	A few expert workers, each fully independent	Thousands of simple workers in lockstep
Does the fits…	One at a time per worker	Tens of thousands at once, batched
Scales by adding…	More CPU cores / more machines	A wider batch on each device
Best when	You can spread buckets over many cores	A single device can be kept saturated

How we test whether the algorithm *scales*

Knowing the model is "embarrassingly parallel" *in theory* is not the same as proving it speeds up *in practice*—disk reads, data shuffling, and coordination overhead can all spoil the ideal. So we built benchmark scripts (`benchmark_scaling.py`, `analyze_scaling.py`) that measure it empirically:

- **Strong scaling.** Hold the problem size *fixed* and increase the number of workers p over $1, 2, 4, 8, 16, 32$. We then compute **speedup** $S(p) = T(1)/T(p)$ and **efficiency** $E(p) = S(p)/p$. Perfect scaling means doubling the workers halves the time ($E \approx 1$). In reality efficiency drops once there are more workers than buckets, or once reading data from disk—not the math—becomes the bottleneck. The question strong scaling answers: *"if I throw more cores at the same job, how much faster does it actually get?"*
- **Weak scaling.** Grow the problem size *in proportion* to the workers (twice the workers, twice the locations). If the algorithm scales well, the wall-clock time stays roughly **flat**. The question: *"if my country/dataset doubles, can I keep the runtime constant by doubling the hardware?"*

- **GPU roofline.** For the GPU we run a separate study (`benchmark_gpu_kernel.py`, `benchmark_gpu_pipeline.py`) timing each stage of the batched solve. It confirms the pipeline is **memory-bandwidth bound**—the GPU spends its time *moving* the data rather than doing arithmetic, which is expected for such lightweight per-fit math and tells us where any future speedups would have to come from.

In short, we are not just asserting the method is parallel; we are running controlled experiments across worker counts and problem sizes, on both CPU and GPU, to **measure how close to ideal the speedup is and where it breaks down.**

What is the expected result or output if I use this? What files do I get?

If you use this tool, you can expect the following outputs:

1. **Climatology table (`daily_climatology.parquet`)**
   For each location and each day of the year: the typical (mean) dewpoint and minimum temperature.
2. **Coefficients table (`llr_coeffs_anomaly_final_direct.parquet`)**
   For each location and each day of the year: the fitted regression coefficients. These are the numbers that tell you, mathematically, how dewpoint is expected to vary as minimum temperature changes, and how yesterday's/more recent conditions affect today.
3. **(Optional/Intermediate) Chunk files**
   Temporary files created when doing parallel calculation for faster or distributed processing.
4. **(If needed) Patch files**
   If the first attempt leaves some days incomplete (missing some combinations of location/day), repair files are created for just those gaps and then merged in.

In practical terms:
- After running this tool, you have all the necessary ingredients to reconstruct a full daily dewpoint record for your area and time period—either filling in missing days or forecasting new ones—by applying these coefficients to available minimum temperature data.

Who should use this?
- Scientists, agricultural experts, and environmental analysts who need complete dewpoint time series for modeling, risk assessment, or gap-filling.